```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np

df = pd.read_excel("/content/drive/MyDrive/ENB2012_data.xlsx")
df.head()
```

	X1	X2	X3	X4	X5	X6	X7	X8	Y1	Y2
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

Next steps: Generate code with df New interactive sheet

```
# Handle missing values
df = df.fillna(df.mean(numeric_only=True))
```

```
df["Heating_Class"] = (df["Y1"] > df["Y1"].median()).astype(int)
```

```
X = df.iloc[:, 0:8]
y = df["Heating_Class"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)

bagging.fit(X_train, y_train)
```

```
▸ BaggingClassifier
▸ estimator: DecisionTreeClassifier
  ▸ DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

y_pred_bag = bagging.predict(X_test)

print("Bagging Accuracy:", accuracy_score(y_test, y_pred_bag))
print(confusion_matrix(y_test, y_pred_bag))
print(classification_report(y_test, y_pred_bag))
```

```
Bagging Accuracy: 0.9935064935064936
[[71  0]
 [ 1 82]]
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        71
           1       1.00      0.99      0.99        83

    accuracy                           0.99       154
   macro avg       0.99      0.99      0.99       154
weighted avg       0.99      0.99      0.99       154
```

```
from sklearn.ensemble import AdaBoostClassifier

boost = AdaBoostClassifier(
    n_estimators=50,
    random_state=42
)

boost.fit(X_train, y_train)
```

```
▾ AdaBoostClassifier
AdaBoostClassifier(random_state=42)
```

```
y_pred_boost = boost.predict(X_test)

print("Boosting Accuracy:", accuracy_score(y_test, y_pred_boost))
print(confusion_matrix(y_test, y_pred_boost))
print(classification_report(y_test, y_pred_boost))
```

```
Boosting Accuracy: 0.987012987012987
[[71  0]
 [ 2 81]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        71
           1       1.00      0.98      0.99        83

    accuracy                           0.99       154
   macro avg       0.99      0.99      0.99       154
weighted avg       0.99      0.99      0.99       154
```

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

estimators = [
    ('knn', KNeighborsClassifier()),
    ('dt', DecisionTreeClassifier())
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

stack.fit(X_train, y_train)
```

```
▸ StackingClassifier
    knn                          dt
  ▸ KNeighborsClassifier       ▸ DecisionTreeClassifier
    final_estimator
  ▸ LogisticRegression
```

```
y_pred_stack = stack.predict(X_test)

print("Stacking Accuracy:", accuracy_score(y_test, y_pred_stack))
print(confusion_matrix(y_test, y_pred_stack))
print(classification_report(y_test, y_pred_stack))
```

```
Stacking Accuracy: 0.987012987012987
[[71  0]
 [ 2 81]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        71
           1       1.00      0.98      0.99        83
```

```
        accuracy                           0.99       154
       macro avg       0.99      0.99      0.99       154
    weighted avg       0.99      0.99      0.99       154
```